Open source languages are on the rise

Microsoft's announcement last year that .NET was going open source attracted considerable attention. This was not a one-off event, but what I see as a general trend in the programming languages community. Java was initially a proprietary language; later, Sun released most of it to the open source community. Languages designed from the ground up, such as Go and Dart (both from Google), TypeScript and F# (both from Microsoft) are also open source. A large number of languages developed in the last two decades are open source. In other words, languages going open source is a clear trend that is here to stay.

Old languages are reborn in new forms

Just like living beings, languages are born, grow up, age and die. What is surprising is the rebirth or rejuvenation of old languages in a new form. The relationship between these new and old language pairs is unmistakable—Elixir and Erlang, Clojure and LISP, Swift and Objective-C, etc. In other words, newer languages such as Elixir, Clojure and Swift render corresponding older languages such as Erlang, LISP and Objective-C in a form that is compatible to the new realities. I won't be surprised if older languages such as APL, Simula, SNOBOL, or even Fortran, are reborn as newer languages.

C continues to be a dominant language

So far we have discussed factors that are changing or emerging. This is a new world with emerging fields like Big Data, cloud computing and the Internet of Things (IoT). The computing world is moving towards increased concurrency and ever shrinking devices. With all these changes, one constant surprises me - C continues to stay relevant in this ever-changing world! With the exploding number of mobile devices that are getting connected to the Internet, the devices are still programmed in embedded C. Low-level utilities such as protocols and device drivers are still written in C. There aren't many rivals for C though Go appears to be an attractive alternative - so if you develop low-level code (e.g., device drivers, virtual machines, protocols, etc), it is better to learn Go. However, the underlying trend is clear—C continues to be relevant and popular.

Other languages to watch out for

The programming languages world is one where so many things are happening that I have FOMO (Fear Of Missing Out). You may be interested in many other languages that I haven't covered here. Some important ones are D from Digital Mars, Ceylon from Red Hat, Chapel from Cray, and Opa from MLState.

So, what language should you learn next?

With this, we have completed our quick safari in the language jungle. It is time to answer the question: "What language should I learn next?"

This question is as difficult to answer as these two questions: "Which book should I read next?" or "Which movie should I watch next?" Why is it difficult to give specific answers to these questions? Because the answer depends on your interests as well as personal preferences! So, here is my answer based on your likely preferences and the overall trends we have discussed so far.

- If you have programmed only in statically typed languages (such as C++ or Java), learn Ruby or Python.
- If you do not know Web programming yet want to learn how to program for the Web, learn JavaScript.
- If you develop systems software and have programmed mostly in C or Assembler, learn Go.
- If you don't know C, learn it!

No matter what kind of programming you do, learn a functional programming language (or at least learn to use closures if the programming language you regularly use allows that).

Also, try using a new language (it's important that it is not a mainstream language) to solve problems you encounter on a day-to-day basis. For example, if you are a game developer and use languages like C++, try using Elm. If your work involves lots of mathematics and you use Fortran, try using languages like Julia or J. If your work involves querying information to find answers using regular expressions, try using Prolog or R. By trying out unusually effective solutions to the problems that you try to solve on a day-to-day basis, you'll be surprised by an 'Aha!' moment that could permanently change the way you think about problem solving! END

Acknowledgement:

I thank Raghu Kalyan Anna for his thoughtful and detailed feedback on an earlier draft of this article.

References

[1] List of languages that compile to JavaScript: *https://github.com/jashkenas/coffeescript/wiki/List-of-languages-that-compile-to-JS*
[2] The Elm language home page: *http://elm-lang.org*
[3] The Hack language home page: *http://hacklang.org*
[4] Java Virtual Machine Support for Non-Java Languages: *http://docs.oracle.com/javase/7/docs/technotes/guides/vm/multiple-language-support.html*
[5] Microsoft Open Sources .NET, saying it will run on Linux and Mac: *http://www.wired.com/2014/11/microsoft-open-sources-net-says-will-run-linux-mac/*
[6] Go language home page: *https://golang.org/*

By: Ganesh Samarthyam

The author is a corporate trainer and independent consultant based in Bengaluru. He is a co-author of the book 'Refactoring for Software Design Smells: Managing Technical Debt' published by Morgan Kaufmann/Elsevier, 2014. You can reach him through his website *www.designsmells.com*.